Array Rotation :

(left rotation)

(leftmost element moves to end)
[1, 2, 3 , 4 , 5] -> rotate this array by 1
        |
[2, 3, 4, 5, 1]

[1, 2, 3, 4, 5] -> rotate this array by 2
       |
[3, 4, 5, 1, 2]

(right rotation)

(rightmost elements move to start)
[1, 2, 3, 4, 5] -> rotate this array by 1
         |
[5, 1, 2, 3, 4]

Int k = gives
Problem : left Rotate this array by k
Array = [1 ,2 , 4, 2, 6,5,8,7,9,11,2] , k = 4

First k elements = [1, 2, 4, 2]

Result = [6, 5, 8,7, 9, 11 , 12, 1, 2, 4, 2]

Perform right rotation
Arr1 = [4,2,3,1,6,7,10, 8,9] , k = 3

Result = [10, 8, 9, 4, 2, 3, 1, 6, 7]

→ Rotation using Reverse operation

Arr = [4, 2, 3, 1, 5], k = 2 (right rotation)
|||||
Result = [1, 5, 4, 2, 3] after performing right rotation

Operation - 1: reverse the whole array from 0 to
[4,2,3,1,5] reversing this array results : [5, 1, 3 , 2 ,4]
0 1 2 3 4 // i = 0 , j = n - 1 where n = size of array

Operation - 2:
I reversed this segment [5, 1] to look like result
Reverse : [1, 5, 3, 2, 4]
// reversing the first segment
// i = 0, j = k - 1

Operation - 3:
I reversed this segment [3, 2, 4] to look like result
Reverse : [1,5, 4, 2, 3]
// reversing the second segment
// i = k, j = n - 1

0 1 2 3 4
[1, 5, 4, 2, 3]
  0 k-1 i   j

```
while(i < j){   // 3 < 3   -> false
    swap(a[i], a[j])
    I++;   // i = 2 —-> i++ —> i = 3
    j–;     // j = 4 —-> j– ->>>>j = 3
}
```

0 1
[5, 1, 3, 2, 4]

i j

i = 0 …….. J = k - 1

[1, 5, 3, 2, 4]

i j

i = k

J = n - 1

Arr = [4, 2, 3, 1, 5] , K = 3

Result : [3, 1, 5, 4, 2]

Linear Search : Checking each element from left to right until our target is found!!

Binary Search : Binary (two) , Search -> khojna

[2, 4, 7, 8, 9 , 10 , 11 , 15, 20]
Algo :

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    // your code goes here

    vector<int> a = {2, 4, 7, 8, 9, 11, 12, 20};
    int target = 20;
    int n = a.size();

    // Perform Binary Search on the array!!
    int low = 0;
    int high = n - 1;

    int flag = 0;

    while(low <= high){
        // find out mid position
        int mid = (low + high) / 2;
        if(a[mid] == target){
```

```
            flag = 1;
            cout << "Target found!!!";
            break;
        }else{
            if(a[mid] < target){ //.  target lies in right
segment
                low = mid + 1 ;
            }else{ //.              target lies in left segment
                high = mid - 1;
            }
        }
    }
    if(flag == 0) cout << "Target not found!!";
}
```

Homework :

Task - 1 : Figure out values of i and j in left rotation

Task - 2 : Write a program to right rotate an array by k

Task - 3 : Write a program of linear search

Task - 4 : Write a program of binary search